

10. Misc.

Contents

Tree Knapsack	2
FFT / NTT	2
Segmented Seive	6
Hilbert Order	6
Wavelet Matrix	7
BCC	9
Blossom	10
Dynamic Bitset	12
Dynamic Suffix Array	12
Dynamic Suffix Array With LCS	14
Generalized Suffix Automaton	19
Suffix Automaton	20
Tarjan SCC / Bridges / Art. Points	21
Static Range Mode Query (sqrt)	23
Lyndon Factorization	24
kinetic_tournament	24
Discrete Root [$x^k = a \pmod m$]	27
Range Progression Sum	28
Count a prime in $n!$	28
Power Tower (Tetration)	28
Stress	29

Tree Knapsack

```
1 vector<int> merge(vector<int> &a, vector<int> &b) {
2     vector<int> res;
3     res.resize(a.size() + b.size() - 1, INF);
4     for (int i = 0; i < a.size(); i++) {
5         if (a[i] == INF) continue;
6         for (int j = 0; j < b.size(); j++) {
7             res[i + j] = min(res[i + j], a[i] + b[j]);
8         }
9     }
10    return res;
11 }
```

FFT / NTT

```
1 namespace FFT {
2     // [1. CONSTANTS] Modulo and its primitive roots for NTT
3     const int mod = 998244353; // 998244353 754974721 167772161
4     const int root = 3; // 3 11 3
5     const int invRoot = 332748118; // 332748118 617706590 55924054
6
7     inline int mul(int x, int y) { return int(x * 1LL * y % mod); }
8     inline int add(int x, int y) { return x + y < mod? x + y: x + y - mod; }
9     inline int sub(int x, int y) { return x - y < 0? x - y + mod: x - y; }
10    int fp(int b, int e) {
11        int res = 1;
12        while(e) {
13            if(e & 1) res = mul(res, b);
14            b = mul(b, b), e >>= 1;
15        }
16        return res;
17    }
18
19    // [3. GENERATOR] O(sqrt(mod) log mod) - Run locally to find
20    // root/invRoot for a new prime mod
21    void primitiveRoot() {
22        int phi = mod - 1;
23        vector<int> fac;
24        for(int i = 2; i * 1LL * i < phi; i++) {
25            if(phi % i == 0) {
26                fac.push_back(i);
27                while(phi % i == 0) phi /= i;
28            }
29        }
30        if(phi > 1) fac.push_back(phi);
31
32        for(int g = 2; g < mod; g++) {
33            for(int pr : fac) if(fp(g, (mod - 1) / pr) == 1)
34                goto bad;
35            cout << "const_int_root=" << g << ";\n";
36            cout << "const_int_invRoot=" << fp(g, mod - 2) << ";\n";
37            return;
38            bad:;
39        }
40        cerr << "404\n";
41    }
42 }
```

```

41 }
42
43 using cd = complex<double>;
44 double pi = acos(-1);
45
46 // [4. FFT CORE] O(N log N) - Used internally by complex multiplication
47 void fft(vector<cd> &a, bool invert) {
48     int n = (int)a.size();
49     for (int i = 1, j = 0; i < n; i++) {
50         j ^= ((1 << __lg(n - 1 ^ j)) - 1) ^ (n - 1);
51         if(i < j)swap(a[i], a[j]);
52     }
53     double ang = pi * (invert ? -1 : 1);
54     for (int len = 1; len < n; len <= 1, ang /= 2) {
55         cd w1(cos(ang), sin(ang));
56         for (int i = 0; i < n; i += len * 2) {
57             cd w(1), u, v;
58             for(int j = 0; j < len; j++) {
59                 u = a[i + j], v = a[i + j + len] * w, w *= w1;
60                 a[i + j] = u + v, a[i + j + len] = u - v;
61             }
62         }
63     }
64     if(invert) for(cd &x : a) x /= n;
65 }
66
67 // [5. FFT MULTIPLY] O(N log N) - Multiplies polynomials without
68 // modulo. Returns exact integers.
69 // Packs 'a' into real and 'b' into imag to save one FFT pass.
70 vector<int64_t> mul(vector<int> const &a, vector<int> const &b) {
71     int N = 1;
72     while (N < a.size() + b.size() - 1) N <= 1;
73     vector<cd> t(N);
74     for(int i = 0; i < a.size(); i++) t[i].real(a[i]);
75     for(int i = 0; i < b.size(); i++) t[i].imag(b[i]);
76     fft(t, false);
77     for(auto &x : t) x *= x;
78     fft(t, true);
79     vector<int64_t> ans(N);
80     for(int i = 0; i < N; i++) ans[i] = (int64_t)round(t[i].imag() / 2.0);
81     return ans;
82 }
83
84 // [6. WILDCARD MATCHING] O(N log N) - Returns starting indices of
85 // pattern 't' in text 's'.
86 // Supports '?' as wildcard. Returns 0-based indices.
87 vector<int> string_matching(string &s, string &t) {
88     if (t.size() > s.size()) return {};
89     int n = s.size(), m = t.size();
90     vector<int> s1(n), s2(n), s3(n);
91     for(int i = 0; i < n; i++) {
92         // assign any non-zero number for non '?'s
93         s1[i] = s[i] == '?' ? 0 : s[i] - 'a' + 1;
94         s2[i] = s1[i] * s1[i];
95         s3[i] = s1[i] * s2[i];
96     }

```

```

97     vector<int> t1(m), t2(m), t3(m);
98     for(int i = 0; i < m; i++) {
99         t1[i] = t[m - i - 1] == '?' ? 0 : t[m - i - 1] - 'a' + 1;
100        t2[i] = t1[i] * t1[i];
101        t3[i] = t1[i] * t2[i];
102    }
103    auto s1t3 = mul(s1, t3);
104    auto s2t2 = mul(s2, t2);
105    auto s3t1 = mul(s3, t1);
106    vector<int> oc;
107    for(int i = m - 1; i < n; i++)
108        if(s1t3[i] - s2t2[i] * 2 + s3t1[i] == 0)
109            oc.push_back(i - m + 1);
110    return oc;
111 }
112
113 // [7. NTT CORE] O(N log N) - Used internally by modular multiplication
114 void ntt(vector<int> &a, bool invert) {
115     int n = (int)a.size();
116     for (int i = 1, j = 0; i < n; i++) {
117         j ^= ((1 << __lg(n - 1 ^ j)) - 1) ^ (n - 1);
118         if(i < j)swap(a[i], a[j]);
119     }
120     for(int len = 2, l2 = 1, w1, w, u, v; len <= n; len <= 1, l2 <= 1) {
121         w1 = fp(invert? invRoot: root, (mod - 1) / len);
122         for(int i = 0; i < n; i += len) {
123             w = 1;
124             for(int j = 0; j < l2; j++) {
125                 u = a[i + j], v = mul(a[i + j + l2], w), w = mul(w, w1);
126                 a[i + j] = add(u, v), a[i + j + l2] = sub(u, v);
127             }
128         }
129     }
130     if (invert) {
131         int n_1 = fp(n, mod - 2);
132         for(int & x : a) x = mul(x, n_1);
133     }
134 }
135
136 // [8. NTT MULTIPLY] O(N log N) - Multiplies polynomials modulo 'mod'.
137 vector<int> mulMod(vector<int> a, vector<int> b) {
138     int N = 1, sz = a.size() + b.size();
139     while (N < a.size() + b.size() - 1) N <= 1;
140     a.resize(N);
141     b.resize(N);
142     ntt(a, false), ntt(b, false);
143     for(int i = 0; i < N; i++)
144         a[i] = int(a[i] * 1LL * b[i] % mod);
145     ntt(a, true);
146     a.resize(sz - 1);
147     return a;
148 }
149
150 // [9. FWHT AND/OR/XOR] O(N log N) - Bitwise convolutions.
151 // N must be power of 2.
152 void fwht_and(vector<ll>& a, bool invert) {

```

```

153     int n = a.size();
154     for (int len = 1; 2 * len <= n; len <= 1) {
155         for (int i = 0; i < n; i += 2 * len) {
156             for (int j = 0; j < len; ++j) {
157                 a[i + j] = (a[i + j] +
158                     (invert? -1: 1) * a[i + j + len] + mod) % mod;
159             }
160         }
161     }
162 }
163 void fwht_or(vector<ll>& a, bool invert) {
164     int n = a.size();
165     for (int len = 1; 2 * len <= n; len <= 1) {
166         for (int i = 0; i < n; i += 2 * len) {
167             for (int j = 0; j < len; ++j) {
168                 a[i + j + len] = (a[i + j + len] +
169                     (invert? -1: 1) * a[i + j] + mod) % mod;
170             }
171         }
172     }
173 }
174 void fwht_xor(vector<ll>& a, bool invert) {
175     int n = a.size();
176     for (int len = 1; 2 * len <= n; len <= 1) {
177         for (int i = 0; i < n; i += 2 * len) {
178             for (int j = 0; j < len; ++j) {
179                 ll u = a[i + j], v = a[i + j + len];
180                 a[i + j] = (u + v) % mod;
181                 a[i + j + len] = (u - v + mod) % mod;
182             }
183         }
184     }
185     if (invert) {
186         ll inv2 = (mod + 1) / 2;
187         ll inv_n = 1;
188         for(int i = 1; i < n; i <= 1)
189             inv_n = inv_n * inv2 % mod;
190         for (ll &x : a) x = x * inv_n % mod;
191     }
192 }
193
194 // [10. CONVOLUTION RUNNER] O(N log N) - Pass fwht_and,
195 // fwht_or, or fwht_xor as 'fun'.
196 // Solves for C[k] = sum(A[i] * B[j]) where i (op) j = k.
197 template<typename F>
198 vector<ll> convolution(vector<ll> a, vector<ll> b, F const &fun) {
199     int n = 1;
200     while (n < max(a.size(), b.size())) n <= 1;
201     a.resize(n), b.resize(n); // Resizes automatically to next power of 2
202     fun(a, false);
203     fun(b, false);
204     for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
205     fun(a, true);
206     return a;
207 }
208 }

```

Segmented Sieve

```
1  vector<int> simple_sieve(int n) {
2      vector<bool> is_prime(n + 1, true);
3      vector<int> primes;
4      if (n >= 0) is_prime[0] = false;
5      if (n >= 1) is_prime[1] = false;
6      for (int i = 2; i <= n; i++) {
7          if (!is_prime[i]) continue;
8          primes.push_back(i);
9          if (1LL * i * i <= n) {
10             for (ll j = 1LL * i * i; j <= n; j += i) {
11                 is_prime[j] = false;
12             }
13         }
14     }
15     return primes;
16 }

17
18 vector<bool> segmented_sieve_mask(ll l, ll r) {
19     if (l > r) return {};
20     int root = sqrtl(r);
21     while (1LL * (root + 1) * (root + 1) <= r) root++;
22     while (1LL * root * root > r) root--;
23     vector<int> primes = simple_sieve(root);
24     vector<bool> is_prime(r - l + 1, true);
25     for (ll p: primes) {
26         ll start = max(p * p, ((l + p - 1) / p) * p);
27         for (ll j = start; j <= r; j += p) {
28             is_prime[j - l] = false;
29         }
30     }
31     for (ll x = l; x <= min(r, 1LL); x++) {
32         is_prime[x - l] = false;
33     }
34     return is_prime;
35 }

36
37 // returns all primes in [l, r]
38 vector<ll> segmented_sieve(ll l, ll r) {
39     vector<bool> is_prime = segmented_sieve_mask(l, r);
40     vector<ll> primes;
41     for (ll i = 0; i < (ll) is_prime.size(); i++) {
42         if (is_prime[i]) primes.push_back(l + i);
43     }
44     return primes;
45 }
```

Hilbert Order

```
1  vector<pair<int64_t, Query> > b1c; // {hilbert_order_id, Query}
2  // pow = __lg(n) + 1, rot = 0
3  function<int64_t(int, int, int, int)> hilbert_order =
4      [&](int x, int y, int pow, int rot) -> int64_t {
5          if (pow == 0) return 0;
```

```

6     int h_pow = 1 << (pow - 1);
7     int seg = ((x < h_pow ? (y < h_pow ? 0 : 3) : (y < h_pow ? 1 : 2)) + rot) & 3;
8     const static int rotate_delta[] = {3, 0, 0, 1};
9     int nx = x & (x ^ h_pow), ny = y & (y ^ h_pow), n_rot = (rot +
10         rotate_delta[seg]) & 3;
11     int64_t sub_square_size = 1LL << (2 * pow - 2), add =
12         hilbert_order(nx, ny, pow - 1, n_rot);
13     return seg * 1LL * sub_square_size + (seg == 1 || seg == 2 ? add :
14         sub_square_size - add - 1);
15 };
16 hilbert_order(l, r, __lg(n) + 1, 0);

```

Wavelet Matrix

```

1  /*
2  * Wavelet Matrix
3  *
4  * Supports:
5  *   - count_less(l, r, x): count numbers < x in a[l..r]
6  *   - sum_less(l, r, x): sum of numbers < x in a[l..r]
7  *   - range_count(l, r, x, y): count numbers in [x, y]
8  *   - range_sum(l, r, x, y): sum of numbers in [x, y]
9  *
10 * Build: O(n log MaxValue)
11 * Query: O(log MaxValue)
12 * Memory: O(n log MaxValue)
13 */
14
15 struct WaveletMatrix {
16     int n, N, LOG;
17     vector<int> bv, mid;
18     vector<long long> ps;
19
20     WaveletMatrix(const vector<int>& orig_a) : n(orig_a.size()),
21         N(orig_a.size() + 1) {
22         if (!n) { LOG = 0; return; }
23         int mx = *max_element(orig_a.begin(), orig_a.end());
24         LOG = mx > 0 ? __lg(mx) + 1 : 1;
25
26         bv.assign(LOG * N, 0);
27         ps.assign((LOG + 1) * N, 0);
28         mid.resize(LOG);
29
30         vector<int> a = orig_a, nxt(n);
31
32         for (int lvl = LOG - 1; lvl >= 0; --lvl) {
33             int pre = lvl * N, off = (lvl + 1) * N;
34             int ones = 0;
35
36             // Merge prefix sum and bit-counting passes
37             for (int i = 0; i < n; i++) {
38                 ps[off + i + 1] = ps[off + i] + a[i];
39                 int b = (a[i] >> lvl) & 1;
40                 bv[pre + i + 1] = bv[pre + i] + b;
41                 ones += b;

```

```

42         }
43
44         int p0 = 0, p1 = mid[lvl] = n - ones;
45         for (int i = 0; i < n; i++) {
46             if ((a[i] >> lvl) & 1) nxt[p1++] = a[i];
47             else nxt[p0++] = a[i];
48         }
49         a = nxt;
50     }
51     for (int i = 0; i < n; i++)
52         ps[i + 1] = ps[i] + a[i];
53 }
54
55 int count_less(int L, int R, int x) {
56     if (L > R || x <= 0 || !LOG) return 0;
57     if (x >> LOG) return R - L + 1;
58
59     R++;
60     int ans = 0;
61
62     for (int lvl = LOG - 1; lvl >= 0; --lvl) {
63         int pre = lvl * N;
64         int oL = bv[pre + L], oR = bv[pre + R];
65         int zL = L - oL, zR = R - oR;
66
67         if ((x >> lvl) & 1) {
68             ans += zR - zL;
69             L = mid[lvl] + oL;
70             R = mid[lvl] + oR;
71         } else {
72             L = zL; R = zR;
73         }
74     }
75     return ans;
76 }
77
78 long long sum_less(int L, int R, int x) {
79     if (L > R || x <= 0 || !LOG) return 0;
80     if (x >> LOG) return ps[LOG * N + R + 1] - ps[LOG * N + L];
81
82     R++;
83     long long ans = 0;
84
85     for (int lvl = LOG - 1; lvl >= 0; --lvl) {
86         int pre = lvl * N;
87         int oL = bv[pre + L], oR = bv[pre + R];
88         int zL = L - oL, zR = R - oR;
89
90         if ((x >> lvl) & 1) {
91             ans += ps[pre + zR] - ps[pre + zL];
92             L = mid[lvl] + oL;
93             R = mid[lvl] + oR;
94         } else {
95             L = zL; R = zR;
96         }
97     }
98     return ans;
99 }

```



```

100
101 int range_count(int L, int R, int x, int y) {
102     if (x > y) return 0;
103     return count_less(L, R, y + 1) - count_less(L, R, x);
104 }
105
106 long long range_sum(int L, int R, int x, int y) {
107     if (x > y) return 0;
108     return sum_less(L, R, y + 1) - sum_less(L, R, x);
109 }
110 };

```

BCC

```

1 struct BCC {
2     int n;
3     vector<int> id;
4     vector<bool> isArt;
5     vector<vector<int>> > tree, g;
6
7     explicit BCC(int n) : n(n), g(n) {
8     }
9
10    void addEdge(int u, int v) {
11        g[u].push_back(v);
12        g[v].push_back(u);
13    }
14
15    void init() {
16        id.assign(n, -1);
17        int m = 0, cnt = 0;
18        vector<int> in(n, -1), low(n, -1), st;
19        st.reserve(n);
20        vector<vector<int>> > bcc(n);
21        function<void(int, int)> dfs = [&](int u, int p) -> void {
22            in[u] = low[u] = cnt++, st.push_back(u);
23            for (int v: g[u]) {
24                if (!~in[v]) {
25                    dfs(v, u), low[u] = min(low[u], low[v]);
26                    if (low[v] >= in[u]) {
27                        int x = -1;
28                        while (x ^ v) x = st.back(), st.pop_back(),
29                            bcc[x].push_back(m);
30                        bcc[u].push_back(m++);
31                    }
32                } else if (v != p) low[u] = min(low[u], in[v]);
33            }
34        };
35        dfs(0, -1);
36        for (int u = 0; u < n; u++) {
37            if (bcc[u].size() == 1) id[u] = bcc[u].front();
38            else id[u] = m++;
39        }
40        tree.assign(m, vector<int>()), isArt.assign(m, false);
41        for (int u = 0; u < n; u++) {

```

```

42         if (bcc[u].size() ^ 1) {
43             isArt[id[u]] = true;
44             for (int v: bcc[u]) {
45                 tree[id[u]].push_back(v);
46                 tree[v].push_back(id[u]);
47             }
48         }
49     }
50 }
51 };

```

Blossom

```

1  /*
2  * Blossom / Edmonds' algorithm for maximum matching in a general undirected graph.
3  *
4  * Usage:
5  *   - Set n and g.
6  *   - g must be an undirected adjacency list.
7  *   - Call findMaximumMatching().
8  *   - Result is stored in match[]:
9  *       match[v] = paired vertex, or -1 if unmatched.
10 *
11 * Complexity:
12 *   -  $O(n^3)$  in the common competitive programming implementation below.
13 *   - Suitable for general graphs with up to a few hundred vertices.
14 *   - Needed only for non-bipartite matching; for bipartite graphs, use Hopcroft-Karp.
15 */
16
17
18 int n;
19 vector<vector<int>> g;
20 vector<int> match, p, base;
21 vector<bool> used, blossom;
22
23 int lca(int a, int b) {
24     vector<bool> vis(n, false);
25     for(;;) {
26         a = base[a], vis[a] = true;
27         if(match[a] == -1) break;
28         a = p[match[a]];
29     }
30     for(;;) {
31         b = base[b];
32         if(vis[b]) return b;
33         b = p[match[b]];
34     }
35 }
36
37 void markPath(int v, int b, int x) {
38     while (base[v] != b) {
39         blossom[base[v]] = blossom[base[match[v]]] = true;
40         p[v] = x;
41         x = match[v];
42         v = p[match[v]];
43     }

```

```

44 }
45
46 int findPath(int src) {
47     used.assign(n, false);
48     p.assign(n, -1);
49     iota(base.begin(), base.end(), 0);
50     queue<int> q;
51     q.push(src);
52     used[src] = true;
53
54     while (!q.empty()) {
55         int v = q.front();
56         q.pop();
57         for (int u: g[v]) {
58             if (base[v] == base[u] || match[v] == u) continue;
59             if (u == src || (match[u] != -1 && p[match[u]] != -1)) {
60                 int cur = lca(v, u);
61                 blossom.assign(n, false);
62                 markPath(v, cur, u);
63                 markPath(u, cur, v);
64                 for (int i = 0; i < n; i++) {
65                     if (blossom[base[i]]) {
66                         base[i] = cur;
67                         if (!used[i]) used[i] = true, q.push(i);
68                     }
69                 }
70             } else if (p[u] == -1) {
71                 p[u] = v;
72                 if (match[u] == -1) return u;
73                 used[match[u]] = true;
74                 q.push(match[u]);
75             }
76         }
77     }
78     return -1;
79 }
80
81 void findMaximumMatching() {
82     match.assign(n, -1);
83     base.resize(n);
84     for (int i = 0; i < n; i++) {
85         if (match[i] == -1) {
86             int v = findPath(i);
87             while (v != -1) {
88                 int pv = p[v], w = match[pv];
89                 match[v] = pv, match[pv] = v, v = w;
90             }
91         }
92     }
93 }
94 void recover(vector<pair<int, int>>& ans) {
95     vector<short> vis(n);
96     for (int i = 0; i < n; i++) {
97         if (!vis[i] && ~match[i]) {
98             vis[i] = vis[match[i]] = 1;
99             ans.emplace_back(i, match[i]);

```

```

100     }
101 }
102 }

```

Dynamic Bitset

```

1 #include <tr2/dynamic_bitset>
2 tr2::dynamic_bitset<long long> bit(N);

```

Dynamic Suffix Array

```

1 #pragma GCC optimize("O3,unroll-loops")
2
3 uint64_t fast_rand() {
4     static uint64_t x = 88172645463325252ULL;
5     x ^= x << 13; x ^= x >> 7; x ^= x << 17;
6     return x;
7 }
8
9 class DynamicSuffixArray {
10     struct Node {
11         int l = 0, r = 0, p = 0, sz = 1;
12         uint64_t y = fast_rand();
13     };
14
15     vector<char> s;
16     vector<Node> tr;
17     int root_ = 0;
18
19     void upd(int x) { if (x) tr[x].sz = 1 + tr[tr[x].l].sz + tr[tr[x].r].sz; }
20
21     int setP(int x, int p) {
22         if (x) tr[x].p = p;
23         return x;
24     }
25
26     int merge(int L, int R) {
27         if (!L || !R) return L + R;
28         if (tr[L].y < tr[R].y) return setP(tr[R].l = merge(L, tr[R].l), R), upd(R), R;
29         setP(tr[L].r = merge(tr[L].r, R), L);
30         return upd(L), L;
31     }
32
33     int getIndex(int x) {
34         if (!x) return 0;
35         int i = tr[tr[x].l].sz + 1;
36         while (tr[x].p) {
37             int p = tr[x].p;
38             if (x == tr[p].r) i += tr[tr[p].l].sz + 1;
39             x = p;
40         }
41         return i;
42     }
43 }

```

```

44 int compare(int id1, int id2, int i) {
45     int cmp = s[id1] - s[id2];
46     return cmp == 0 ? i - getIndex(id2 - 1) : cmp;
47 }
48
49 void split(int x, int id, int i, int& L, int& R) {
50     if (!x) { L = R = 0; return; }
51     if (compare(id, x, i) < 0) {
52         split(tr[x].l, id, i, L, tr[x].l);
53         setP(tr[x].l, x);
54         R = x;
55     } else {
56         split(tr[x].r, id, i, tr[x].r, R);
57         setP(tr[x].r, x);
58         L = x;
59     }
60     upd(x);
61 }
62
63 int insert(int x, int id, int i1) {
64     if (!x) return id;
65     if (tr[x].y < tr[id].y) {
66         split(x, id, i1, tr[id].l, tr[id].r);
67         setP(tr[id].l, id);
68         setP(tr[id].r, id);
69         return upd(id), id;
70     }
71     if (compare(id, x, i1) < 0) setP(tr[x].l = insert(tr[x].l, id, i1), x);
72     else setP(tr[x].r = insert(tr[x].r, id, i1), x);
73     tr[x].sz++;
74     return x;
75 }
76
77 public:
78     DynamicSuffixArray(int max_capacity = 1000000) {
79         s.reserve(max_capacity + 10);
80         tr.reserve(max_capacity + 10);
81         s.push_back(0);
82         tr.emplace_back();
83         tr[0].sz = 0;
84     }
85
86     void push_front(char c) {
87         s.push_back(c);
88         tr.emplace_back();
89         int id = size();
90         root_ = insert(root_, id, getIndex(id - 1));
91     }
92
93     char pop_front() {
94         if (size() <= 0) return 0;
95         int id = size(), p = tr[id].p, tmp = merge(tr[id].l, tr[id].r);
96         setP(tmp, p);
97         if (p) {
98             (tr[p].l == id ? tr[p].l : tr[p].r) = tmp;
99             while (p) tr[p].sz--, p = tr[p].p;

```

```

100         } else root_ = tmp;
101
102         tr.pop_back();
103         char c = s.back();
104         s.pop_back();
105         return c;
106     }
107     // count number of occurrences
108     int match(const string& q, int x = -1, bool L = false, bool R = false) {
109         if (x == -1) x = root_;
110         if (!x) return 0;
111         if (L && R) return tr[x].sz;
112         for (size_t i = 0; i < q.size(); ++i) {
113             if (q[i] > s[x - i]) return match(q, tr[x].r, false, R);
114             if (q[i] < s[x - i]) return match(q, tr[x].l, L, false);
115         }
116         return 1 + match(q, tr[x].l, L, true) + match(q, tr[x].r, true, R);
117     }
118
119     int size() { return (int)s.size() - 1; }
120     char at(int i) { return s[size() - i]; }
121 };

```

Dynamic Suffix Array With LCS

```

1  static uint64_t rsd = chrono::steady_clock::now().time_since_epoch().count();
2  inline uint64_t fast_rand() {
3      rsd ^= rsd << 13; rsd ^= rsd >> 7; rsd ^= rsd << 17;
4      return rsd;
5  }
6
7  class DynamicSuffixArray {
8      using u64 = uint64_t; using u128 = __int128_t;
9
10     // Secure prime modulo to prevent anti-hash WA
11     static constexpr u64 MOD = (1ULL << 61) - 1;
12     static constexpr u64 BASE = 11995408973635179863ULL % MOD;
13
14     static inline u64 add(u64 a, u64 b) { return a + b >= MOD ? a + b - MOD : a + b; }
15     static inline u64 sub(u64 a, u64 b) { return a >= b ? a - b : a + MOD - b; }
16     static inline u64 mul(u64 a, u64 b) {
17         u128 r = (u128)a * b;
18         u64 res = (u64)(r >> 61) + (u64)(r & MOD);
19         return res >= MOD ? res - MOD : res;
20     }
21
22     struct Node {
23         int l = 0, r = 0, p = 0, sz = 1;
24         int mn = 0, mx = 0;
25         u64 y; Node() : y(fast_rand()) {}
26     };
27     vector<char> s; vector<Node> tr; vector<u64> hp, pw;
28     int root_ = 0;
29
30     void ensure_powers(int n) {
31         while ((int)pw.size() <= n)

```

```

32         pw.push_back(mul(pw.back(), BASE));
33     }
34
35     inline void upd(int x) {
36         if (!x) return;
37         tr[x].sz = 1 + tr[tr[x].l].sz + tr[tr[x].r].sz;
38         tr[x].mn = tr[x].mx = x;
39         if (tr[x].l) {
40             tr[x].mn = min(tr[x].mn, tr[tr[x].l].mn);
41             tr[x].mx = max(tr[x].mx, tr[tr[x].l].mx);
42         }
43         if (tr[x].r) {
44             tr[x].mn = min(tr[x].mn, tr[tr[x].r].mn);
45             tr[x].mx = max(tr[x].mx, tr[tr[x].r].mx);
46         }
47     }
48
49     inline int setP(int x, int p) {
50         if (x) tr[x].p = p; return x;
51     }
52
53     int merge(int L, int R) {
54         if (!L || !R) return L + R;
55         if (tr[L].y < tr[R].y)
56             return setP(tr[R].l = merge(L, tr[R].l), R), upd(R), R;
57         return setP(tr[L].r = merge(tr[L].r, R), L), upd(L), L;
58     }
59
60     int getIndex(int x) {
61         if (!x) return 0;
62         int i = tr[tr[x].l].sz + 1;
63         while (tr[x].p) {
64             int p = tr[x].p;
65             if (x == tr[p].r) i += tr[tr[p].l].sz + 1;
66             x = p;
67         }
68         return i;
69     }
70
71     int compare(int id1, int id2, int i) {
72         int cmp = s[id1] - s[id2];
73         return cmp == 0 ? i - getIndex(id2 - 1) : cmp;
74     }
75
76     void split(int x, int id, int i, int &L, int &R) {
77         if (!x) { L = R = 0; return; }
78         if (compare(id, x, i) < 0) {
79             split(tr[x].l, id, i, L, tr[x].l);
80             setP(tr[x].l, x);
81             R = x;
82         } else {
83             split(tr[x].r, id, i, tr[x].r, R);
84             setP(tr[x].r, x);
85             L = x;
86         } upd(x);
87     }

```

```

88
89 int insert(int x, int id, int i1) {
90     if (!x) return id;
91     if (tr[x].y < tr[id].y) {
92         split(x, id, i1, tr[id].l, tr[id].r);
93         setP(tr[id].l, id);
94         setP(tr[id].r, id);
95         return upd(id), id;
96     }
97     if (compare(id, x, i1) < 0) setP(tr[x].l = insert(tr[x].l, id, i1), x);
98     else setP(tr[x].r = insert(tr[x].r, id, i1), x);
99     tr[x].sz++; return x;
100 }
101
102 inline u64 str_hash(const vector<u64> &h, int l, int len) const {
103     return sub(h[l + len], mul(h[l], pw[len]));
104 }
105
106 vector<u64> make_hash(const string &q) const {
107     vector<u64> h(q.size() + 1);
108     for (size_t i = 0; i < q.size(); ++i)
109         h[i + 1] = add(mul(h[i], BASE), (unsigned char)q[i]);
110     return h;
111 }
112
113 int lcp_query_node(const string &q, const vector<u64> &qh, int qi, int x) const {
114     int lo = 0, hi = min((int)q.size() - qi, x);
115     while (lo < hi) {
116         int mid = (lo + hi + 1) >> 1;
117         u64 h_q = str_hash(qh, qi, mid);
118         u64 h_s = sub(hp[x], hp[x - mid]);
119         if (mul(h_q, pw[x - mid]) == h_s) lo = mid;
120         else hi = mid - 1;
121     }
122     return lo;
123 }
124
125 inline int lcp_query_node_fast(const string &q, const vector<u64> &qh,
126     int qi, int x) const {
127     return !x ? 0 : lcp_query_node(q, qh, qi, x);
128 }
129
130 int compare_prefix(const string &q, const vector<u64> &qh, int qi, int x) const {
131     int l = lcp_query_node(q, qh, qi, x);
132     if (l == (int)q.size() - qi) return 0; // q is prefix of x
133     if (l == x) return 1; // x is prefix of q => q > x
134     return q[qi + l] < s[x - l] ? -1 : 1;
135 }
136
137 int compare_query_node(const string &q, const vector<u64> &qh, int qi, int x)
138 const {
139     int l = lcp_query_node(q, qh, qi, x);
140     int qlen = (int)q.size() - qi, slen = x;
141     if (l == qlen && l == slen) return 0;
142     if (l == qlen) return -1;
143     if (l == slen) return 1;

```



```

144     return q[qi + 1] < s[x - 1] ? -1 : 1;
145 }
146
147 void split_lower(int x, const string &q, const vector<u64> &qh, int qi,
148     int &L, int &R) {
149     if (!x) { L = R = 0; return; }
150     if (compare_prefix(q, qh, qi, x) > 0) { // q > suffix -> suffix belongs in L
151         split_lower(tr[x].r, q, qh, qi, tr[x].r, R);
152         setP(tr[x].r, x); L = x;
153     } else {
154         split_lower(tr[x].l, q, qh, qi, L, tr[x].l);
155         setP(tr[x].l, x); R = x;
156     }
157     upd(x);
158 }
159
160 void split_upper(int x, const string &q, const vector<u64> &qh, int qi, int &L,
161     int &R) {
162     if (!x) { L = R = 0; return; }
163     // q >= suffix (includes prefix match) -> belongs in L
164     if (compare_prefix(q, qh, qi, x) >= 0) {
165         split_upper(tr[x].r, q, qh, qi, tr[x].r, R);
166         setP(tr[x].r, x); L = x;
167     } else {
168         split_upper(tr[x].l, q, qh, qi, L, tr[x].l);
169         setP(tr[x].l, x); R = x;
170     }
171     upd(x);
172 }
173
174 struct MatchResult { int cnt, first_occ, last_occ; };
175
176 MatchResult get_matches(const string &q) {
177     if (q.empty() || size() == 0) return {0, -1, -1};
178     ensure_powers(max(size(), (int)q.size()));
179     int L = 0, M = 0, R = 0, tmp = 0;
180     vector<u64> qh = make_hash(q);
181     split_lower(root_, q, qh, 0, L, tmp);
182     split_upper(tmp, q, qh, 0, M, R);
183     MatchResult res = {0, -1, -1};
184     if (M) {
185         res.cnt = tr[M].sz;
186         res.first_occ = size() - tr[M].mx;
187         res.last_occ = size() - tr[M].mn;
188     }
189     root_ = merge(L, merge(M, R));
190     return res;
191 }
192
193 pair<int, int> pred_succ(const string &q, const vector<u64> &qh, int qi) const {
194     int x = root_, pred = 0, succ = 0;
195     while (x) {
196         int cmp = compare_query_node(q, qh, qi, x);
197         if (cmp <= 0) {
198             succ = x; x = tr[x].l;
199         } else {

```

```

200         pred = x; x = tr[x].r;
201     }
202 }
203 return {pred, succ};
204 }
205
206 public:
207     DynamicSuffixArray(int max_capacity = 1000000) {
208         s.reserve(max_capacity + 10);
209         tr.reserve(max_capacity + 10);
210         hp.reserve(max_capacity + 10);
211         pw.reserve(max_capacity + 10);
212         s.push_back(0); tr.emplace_back();
213         tr[0].sz = 0; tr[0].mn = INT_MAX; tr[0].mx = -1;
214         hp.push_back(0); pw.push_back(1);
215     }
216
217     void push_front(char c) {
218         int old_size = size();
219         s.push_back(c); tr.emplace_back();
220         int id = size(); tr[id].mn = tr[id].mx = id;
221         root_ = insert(root_, id, getIndex(id - 1));
222         ensure_powers(old_size + 1);
223         hp.push_back(add(mul((u64)(unsigned char)c, pw[old_size]), hp[old_size]));
224     }
225
226     char pop_front() {
227         if (size() <= 0) return 0;
228         int id = size(), p = tr[id].p, tmp = merge(tr[id].l, tr[id].r);
229         setP(tmp, p);
230         if (p) {
231             (tr[p].l == id ? tr[p].l : tr[p].r) = tmp;
232             while (p) {
233                 upd(p);
234                 p = tr[p].p;
235             }
236         } else
237             root_ = tmp;
238
239         char c = s.back();
240         tr.pop_back(); s.pop_back(); hp.pop_back();
241         return c;
242     }
243
244     int count_occurrences(const string &q) { return get_matches(q).cnt; }
245     int find_first(const string &q) { return get_matches(q).first_occ; }
246     int find_last(const string &q) { return get_matches(q).last_occ; }
247
248     int lcs(const string &q) {
249         if (q.empty() || size() == 0) return 0;
250         ensure_powers(max(size(), (int)q.size()));
251         vector<u64> qh = make_hash(q);
252         int best_len = 0, best_pos = 0;
253         for (int i = 0; i < (int)q.size(); ++i) {
254             auto [pred, succ] = pred_succ(q, qh, i);
255             if (pred) {

```

```

256         int len = lcp_query_node(q, qh, i, pred);
257         if (len > best_len) { best_len = len; best_pos = i; }
258     }
259     if (succ) {
260         int len = lcp_query_node(q, qh, i, succ);
261         if (len > best_len) { best_len = len; best_pos = i; }
262     }
263 }
264 // To restore the substring, you can use: q.substr(best_pos, best_len)
265 return best_len;
266 }
267 int size() const { return (int)s.size() - 1; }
268 char at(int i) const { return s[size() - i]; }
269 };

```

Generalized Suffix Automaton

```

1  struct generalized_sam {
2      struct state : map<int, int> {
3          int fail = -1, len{}; cnt = 0;
4          bool ed = false;
5      };
6      vector<state> tr;
7      int max_len = 0;
8
9      explicit generalized_sam() : tr(1) {}
10
11     int add(int c, int last) {
12         if (tr[last].count(c)) {
13             int q = tr[last][c];
14             if (tr[last].len + 1 == tr[q].len) return q;
15             int clone = int(tr.size());
16             tr.emplace_back(tr[q]);
17             tr[clone].cnt = 0, tr[clone].len = tr[last].len + 1;
18             max_len = max(max_len, tr[clone].len);
19             int p = last;
20             while (~p && tr[p][c] == q) tr[p][c] = clone, p = tr[p].fail;
21             tr[q].fail = clone;
22             return clone;
23         }
24
25         int x = int(tr.size());
26         tr.emplace_back();
27         tr[x].len = tr[last].len + 1;
28         max_len = max(max_len, tr[x].len);
29         int p = last;
30         while (~p && !tr[p].count(c)) tr[p][c] = x, p = tr[p].fail;
31         if (p == -1) tr[x].fail = 0;
32         else {
33             int q = tr[p][c];
34             if (tr[p].len + 1 == tr[q].len) tr[x].fail = q;
35             else {
36                 int y = int(tr.size());
37                 tr.emplace_back(tr[q]);
38                 tr[y].cnt = 0, tr[y].len = tr[p].len + 1;

```

```

39         max_len = max(max_len, tr[y].len);
40
41         while (~p && tr[p][c] == q) tr[p][c] = y, p = tr[p].fail;
42         tr[x].fail = tr[q].fail = y;
43     }
44 }
45 return x;
46 }
47
48 void insert(const string& s) {
49     int last = 0;
50     for (auto i : s) {
51         last = add(i, last);
52         tr[last].cnt++;
53     }
54     tr[last].ed = true;
55 }
56
57 void init() {
58     vector b(max_len + 1, vector(0, 0));
59     for (int i = 0; i < tr.size(); ++i) b[tr[i].len].push_back(i);
60     for (int l = max_len; l >= 1; --l) {
61         for (int u : b[l]) {
62             if (~tr[u].fail) {
63                 tr[tr[u].fail].cnt += tr[u].cnt;
64                 if (tr[u].ed) tr[tr[u].fail].ed = true;
65             }
66         }
67     }
68 }
69 };

```

Suffix Automaton

```

1  struct suffix_automaton {
2      struct node {
3          int len = 0, link = -1, fpos = 0, lpos = 0 ;
4          map<char, int> nxt; // array<int,26> nxt;
5          node(int len = 0, int link = -1, int fpos = 0)
6              : len(len), link(link), fpos(fpos) { lpos = fpos ;}
7      };
8
9      #define len(v) sa[v].len
10     #define link(v) sa[v].link
11     #define fpos(v) sa[v].fpos
12     #define lpos(v) sa[v].lpos
13     #define nxt(v) sa[v].nxt
14
15     vector<node> sa;
16     vector<vector<int>>> g;
17     int sz = 1, last = 0;
18
19     suffix_automaton() {
20         sa.emplace_back();
21     }
22     void extend(char c) {

```

```

23     int curr = sz++;
24     sa.emplace_back(len(last) + 1, -1, len(last) + 1);
25     int p = last;
26     while (p != -1 && !nxt(p).count(c)) {
27         nxt(p)[c] = curr;
28         p = link(p);
29     }
30     if (p == -1) {
31         link(curr) = 0;
32     } else {
33         int q = nxt(p)[c];
34         if (len(q) == len(p) + 1) {
35             link(curr) = q;
36         } else {
37             int clone = sz++;
38             sa.push_back(sa[q]);
39             len(clone) = len(p) + 1;
40             while (p != -1 && nxt(p).count(c) && nxt(p)[c] == q) {
41                 nxt(p)[c] = clone;
42                 p = link(p);
43             }
44             link(q) = link(curr) = clone;
45         }
46     }
47     last = curr;
48 }
49
50
51 void prop() { // faster than build_tree and less memory
52     int mxln = 0 ;
53     for (auto &u : sa) mxln = max(mxln , u.len) ;
54     vector<int> acc(mxln + 1) , order(sz) ;
55     for (auto &u : sa) acc[u.len]++;
56     for (int i = 1 ; i <= mxln ; i++) acc[i] += acc[i - 1] ;
57     for (int i = sz - 1 ; i >= 0 ; i--) order[--acc[sa[i].len]] = i ;
58     for (int i = sz - 1 ; i > 0 ; i--) {
59         int node = order[i] , lnk = link(node) ;
60         sa[lnk].lpos = max(sa[lnk].lpos , sa[node].lpos) ;
61     }
62 }
63 int move (int v , char &c) { return (nxt(v).count(c) ? nxt(v)[c] : 0); }
64 void move(int &v, int &len, char &c) {
65     while (v > 0 && !nxt(v).count(c)) v = link(v) , len = len(v);
66     if (nxt(v).count(c)) v = nxt(v)[c] , len++;
67     else v = len = 0;
68 }
69 };
70 string s = "ababa";
71 suffix_automaton sam;
72 for (char c : s) sam.extend(c);
73 sam.prop();

```

Tarjan SCC / Bridges / Art. Points

```

1 struct TarjanSCC {

```

```

2   bool directed;
3   int n, timer, sccCount;
4   vector<int> in, low, sccId;
5   vector<bool> inStack, isArticulation;
6   stack<int> st;
7   vector<pair<int, int>> bridges;
8   vector<vector<int>> graph, sccs, dag;
9
10  explicit TarjanSCC(int n, bool directed = false) :
11      directed(directed), n(n), timer(0), sccCount(0) {
12      graph.resize(n);
13      in = low = sccId = vector(n, -1);
14      inStack = isArticulation = vector(n, false);
15  }
16
17  void addEdge(int u, int v) {
18      graph[u].push_back(v);
19      if(!directed) graph[v].push_back(u);
20  }
21
22  void dfs(int u, int parent = -1) {
23      in[u] = low[u] = ++timer;
24      st.push(u), inStack[u] = true;
25
26      int children = 0;
27      for (int v : graph[u]) if(directed || v != parent) {
28          if (in[v] == -1) {
29              children++;
30              dfs(v, u);
31
32              if(low[v] > in[u]) bridges.emplace_back(u, v);
33              if(parent != -1 && low[v] >= in[u]) isArticulation[u] = true;
34              low[u] = min(low[u], low[v]);
35          }
36          else if(inStack[v])
37              low[u] = min(low[u], in[v]);
38      }
39      if(parent == -1 && children > 1) isArticulation[u] = true;
40
41      if (low[u] == in[u]) {
42          sccs.emplace_back();
43          int v;
44          do {
45              v = st.top(), st.pop(), inStack[v] = false;
46              sccId[v] = sccCount;
47              sccs.back().push_back(v);
48          } while (v != u);
49          ++sccCount;
50      }
51  }
52
53  void init() { for(int i = 0; i < n; ++i) if(in[i] == -1) dfs(i); }
54
55  void buildDAG() {
56      dag.assign(sccCount, {});
57      set<pair<int, int>> edgeSet;

```

```

58     for (int u = 0, su, sv; u < n; ++u) {
59         su = sccId[u];
60         for(int v : graph[u]) {
61             sv = sccId[v];
62             if(su != sv && edgeSet.insert({su, sv}).second)
63                 dag[su].push_back(sv);
64         }
65     }
66 }
67 }
68 };

```

Static Range Mode Query (sqrt)

```

1  template <typename T>
2  struct StaticRangeModeQuery {
3      int n, S, B;
4      vector<T> val;
5      vector<int> a, pos, pos_inv, st;
6      vector<pair<int, int>> ans;
7
8      StaticRangeModeQuery() = default;
9      explicit StaticRangeModeQuery(const vector<T> &arr) {
10         n = arr.size();
11         S = max<int>(sqrt(n), 1);
12         B = (n + S - 1) / S;
13
14         val = arr;
15         sort(val.begin(), val.end());
16         val.erase(unique(val.begin(), val.end()), val.end());
17         int V = val.size();
18
19         a.resize(n), st.assign(V + 1, 0);
20         for (int i = 0; i < n; ++i)
21             ++st[a[i] = lower_bound(val.begin(), val.end(), arr[i]) - val.begin()];
22
23         for (int i = 0; i < V; ++i) st[i + 1] += st[i];
24
25         pos.resize(n), pos_inv.resize(n);
26         for (int i = n; i--;)
27             pos[pos_inv[i] = --st[a[i]]] = i;
28
29         ans.assign((B + 1) * (B + 1), {0, 0});
30         vector<int> cnt(V);
31         for (int l = 0; l <= B; ++l) {
32             cnt.assign(V, 0);
33             pair<int, int> cur{0, 0};
34             for (int r = l + 1; r <= B; ++r) {
35                 for (int i = (r - 1) * S, end = min(n, i + S); i < end; ++i) {
36                     pair<int, int> cand{++cnt[a[i]], a[i]};
37                     if (cand > cur) cur = cand;
38                 }
39                 ans[l * (B + 1) + r] = cur;
40             }
41         }

```

```

42     }
43
44     pair<T, int> query(int l, int r) const {
45         int lb = (l + S - 1) / S, rb = r / S;
46         auto [freq, res] = ans[lb * (B + 1) + rb];
47
48         for (int i = l, end = min(r, lb * S); i < end; ++i) {
49             int v = a[i], idx = pos_inv[i];
50             while (idx + freq < st[v + 1] && pos[idx + freq] < r) ++freq, res = v;
51         }
52         for (int i = r, end = max(l, rb * S); i-- > end;) {
53             int v = a[i], idx = pos_inv[i];
54             while (idx - freq >= st[v] && pos[idx - freq] >= l) ++freq, res = v;
55         }
56         return {val[res], freq};
57     }
58
59     pair<T, int> operator()(int l, int r) const { return query(l, r); }
60 };

```

Lyndon Factorization

```

1  // returns an array of end points of each word (exclusive)
2  // s = [0,v[0]] + [v[0],v[1]] + ...
3  vector<int> duval(vector<int> &s){
4      int n = sz(s);
5      vector<int> v;
6      for(int i = 0; i < n; ){
7          int j = i + 1, k = i;
8          while(j < n && s[k] <= s[j]){
9              if(s[k] == s[j]) j++, k++;
10             else if(s[k] < s[j]) j++, k = i;
11         }
12         while(i <= k){
13             i += j - k;
14             v.push_back(i);
15         }
16     }
17     return v;
18 }

```

kinetic_tournament

```

1  // Suppose that you have an array containing pairs of nonnegative integers,
2  // A[i] and B[i]. You also have a global parameter T, corresponding to the
3  // "temperature" of the data structure. Your goal is to support the following
4  // queries on this data:
5  //
6  //   - update(i, a, b): set A[i] = a and B[i] = b
7  //   - query(s, e): return min{s <= i <= e} A[i] * T + B[i]
8  //   - heaten(new_temp): set T = new_temp
9  //       [precondition: new_temp >= current value of T]
10 //
11 // (For simplicity, we set A[i] = 0 and B[i] = LLONG_MAX for uninitialized

```



```

12 // entries, which should not change the query results.)
13 //
14 // This allows you to essentially do arbitrary lower convex hull queries on a
15 // collection of lines, as well as any contiguous subcollection of those lines.
16 // This is more powerful than standard convex hull tricks and related data
17 // structures (Li-Chao Segment Tree) for three reasons:
18 //
19 // - You can arbitrarily remove/edit lines, not just add them.
20 // - Dynamic access to any subinterval of lines, which lets you avoid costly
21 //   merge small-to-large operations in some cases.
22 // - Easy to reason about and implement from scratch, unlike dynamic CHT.
23 //
24 // The tradeoff is that you can only query sequential values (temperature is
25 // only allowed to increase) for amortization reasons, but this happens to be
26 // a fairly common case in many problems.
27 //
28 // Time complexity:
29 //
30 // - query:  $O(\log n)$ 
31 // - update:  $O(\log n)$ 
32 // - heaten:  $O(\log^2 n)$  [amortized]
33 //
34
35 #include <bits/stdc++.h>
36 using namespace std;
37
38 template <typename T = int64_t>
39 class kinetic_tournament {
40     const T INF = numeric_limits<T>::max();
41     typedef pair<T, T> line;
42
43     size_t n;           // size of the underlying array
44     T temp;             // current temperature
45     vector<line> st;     // tournament tree
46     vector<T> melt;     // melting temperature of each subtree
47
48     inline T eval(const line& ln, T t) {
49         return ln.first * t + ln.second;
50     }
51
52     inline bool cmp(const line& line1, const line& line2) {
53         auto x = eval(line1, temp);
54         auto y = eval(line2, temp);
55         if (x != y) return x < y;
56         return line1.first < line2.first;
57     }
58
59     T next_isect(const line& line1, const line& line2) {
60         if (line1.first > line2.first) {
61             T delta = eval(line2, temp) - eval(line1, temp);
62             T delta_slope = line1.first - line2.first;
63             assert(delta > 0);
64             T mint = temp + (delta - 1) / delta_slope + 1;
65             return mint > temp ? mint : INF; // prevent overflow
66         }
67         return INF;
68     }

```

```

69
70 void recompute(size_t lo, size_t hi, size_t node) {
71     if (lo == hi || melt[node] > temp) return;
72
73     size_t mid = (lo + hi) / 2;
74     recompute(lo, mid, 2 * node + 1);
75     recompute(mid + 1, hi, 2 * node + 2);
76
77     auto line1 = st[2 * node + 1];
78     auto line2 = st[2 * node + 2];
79     if (!cmp(line1, line2))
80         swap(line1, line2);
81     st[node] = line1;
82
83     melt[node] = min(melt[2 * node + 1], melt[2 * node + 2]);
84     if (line1 != line2) {
85         T t = next_isect(line1, line2);
86         assert(t > temp);
87         melt[node] = min(melt[node], t);
88     }
89 }
90
91 void update(size_t i, T a, T b, size_t lo, size_t hi, size_t node) {
92     if (i < lo || i > hi) return;
93     if (lo == hi) {
94         st[node] = {a, b};
95         return;
96     }
97     size_t mid = (lo + hi) / 2;
98     update(i, a, b, lo, mid, 2 * node + 1);
99     update(i, a, b, mid + 1, hi, 2 * node + 2);
100    melt[node] = 0;
101    recompute(lo, hi, node);
102 }
103
104 T query(size_t s, size_t e, size_t lo, size_t hi, size_t node) {
105     if (hi < s || lo > e) return INF;
106     if (s <= lo && hi <= e) return eval(st[node], temp);
107     size_t mid = (lo + hi) / 2;
108     return min(query(s, e, lo, mid, 2 * node + 1),
109               query(s, e, mid + 1, hi, 2 * node + 2));
110 }
111
112 public:
113     // Constructor for a kinetic tournament, takes in the size n of the
114     // underlying arrays a[..], b[..] as input.
115     kinetic_tournament(size_t size) : n(size), temp(0) {
116         assert(size > 0);
117         size_t seg_size = ((size_t) 2) << (64 - __builtin_clzll(n - 1));
118         st.resize(seg_size, {0, INF});
119         melt.resize(seg_size, INF);
120     }
121
122     // Sets A[i] = a, B[i] = b.
123     void update(size_t i, T a, T b) {
124         update(i, a, b, 0, n - 1, 0);

```

```

125     }
126
127     // Returns min{s <= i <= e} A[i] * T + B[i].
128     T query(size_t s, size_t e) {
129         return query(s, e, 0, n - 1, 0);
130     }
131
132     // Increases the internal temperature to new_temp.
133     void heaten(T new_temp) {
134         assert(new_temp >= temp);
135         temp = new_temp;
136         recompute(0, n - 1, 0);
137     }
138 };

```

Discrete Root [$x^k = a \pmod{m}$]

```

1 // returns any or all numbers x such that  $x^k = a \pmod{m}$ 
2 // existence:  $a = 0$  is trivial, and if  $a > 0$ :  $a^{(\phi(m) / \gcd(k, \phi(m)))} \equiv 1 \pmod{m}$ 
3 // if solution exists, then number of solutions =  $\gcd(k, \phi(m))$ .
4 // here m is prime, but it will work for any integer which has a primitive root
5 int discrete_root(int k, int a, int m) {
6     if (a == 0) return 1;
7     int g = primitive_root(m);
8     int phi = m - 1; // m is prime
9     // run baby step-giant step
10    int sq = (int) sqrt(m + .0) + 1;
11    vector<pair<int, int>> dec(sq);
12    for (int i = 1; i <= sq; ++i) dec[i - 1] =
13        make_pair(power(g, 1LL * i * sq % phi * k % phi, m), i);
14    sort(dec.begin(), dec.end());
15    int any_ans = -1;
16    for (int i = 0; i < sq; ++i) {
17        int my = power(g, 1LL * i * k % phi, m) * 1LL * a % m;
18        auto it = lower_bound(dec.begin(), dec.end(), make_pair(my, 0));
19        if (it != dec.end() && it->first == my) {
20            any_ans = it->second * sq - i;
21            break;
22        }
23    }
24    if (any_ans == -1) return -1; //no solution
25    // for any answer
26    int delta = (m - 1) / __gcd(k, m - 1);
27    return power(g, any_ans % delta, m);
28    // // for all possible answers
29    // int delta = (m - 1) / __gcd(k, m - 1);
30    // vector<int> ans;
31    // for (int cur = any_ans % delta; cur < m-1; cur += delta)
32    //     ans.push_back(power(g, cur, m));
33    // sort(ans.begin(), ans.end());
34    // // assert(ans.size() == __gcd(k, m - 1))
35    // return ans;
36 }

```

Range Progression Sum

```
1 struct tag {
2     ll a = 0, d = 0;
3     void apply(const tag &p) {
4         a += p.a; d += p.d;
5     }
6 };
7 struct info {
8     ll sum = 0;
9     info(ll sum = 0) : sum(sum) {}
10    void apply(const tag &lz, int lx, int rx) {
11        ll len = rx - lx + 1;
12        ll first = lz.a + lz.d * lx;
13        ll last = lz.a + lz.d * rx;
14        sum += (first + last) * len / 2;
15    }
16    friend info operator+(const info &l, const info &r) {
17        return {l.sum + r.sum};
18    }
19 };
20
21 // add A, A+D, A+2D, ... to [l,r]
22 st.applyRange(l, r, {
23     A - 1LL * D * l,
24     D
25 });
```

Count a prime in n!

```
1 ll count_p_in_nfact(ll p, ll n) {
2     ll res = 0, q = p;
3     while (q <= n) res += n / q, q *= p;
4     return res;
5 }
```

Power Tower (Tetration)

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3 struct chash {
4     const int RANDOM = (ll)(make_unique<char>().get()) ^
5         chrono::high_resolution_clock::now().time_since_epoch().count();
6
7     static unsigned long long hash_f(unsigned long long x) {
8         x += 0x9e3779b97f4a7c15;
9         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
10        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
11        return x ^ (x >> 31);
12    }
13    static unsigned hash_combine(unsigned a, unsigned b) {
14        return a * 31 + b;
15    }
16    int operator()(int x) const {
```

```

17     return hash_f(x)^RANDOM;
18 }
19 };
20 gp_hash_table<ll, ll, chash> phi_cache;
21 ll get_phi(ll n) {
22     if (phi_cache.find(n) != phi_cache.end()) return phi_cache[n];
23     ll ans = n, m = n;
24     for (ll i = 2; i * i <= m; i++) {
25         if (m % i == 0) {
26             while (m % i == 0) m /= i;
27             ans -= ans / i; // Condensed standard phi formula
28         }
29     }
30     if (m > 1) ans -= ans / m;
31     return phi_cache[n] = ans;
32 }
33 ll ext_mod(__int128_t x, ll m) {
34     return x < m ? (ll)x : (ll)(x % m) + m;
35 }
36 ll ext_pow(ll base, ll exp, ll mod) {
37     base = ext_mod(base, mod);
38     ll res = ext_mod(1, mod);
39     while (exp > 0) {
40         if (exp & 1) res = ext_mod((__int128_t)res * base, mod);
41         base = ext_mod((__int128_t)base * base, mod);
42         exp >>= 1;
43     }
44     return res;
45 }
46 ll solve_tower(const vector<ll> &a, int l, int r, ll m) {
47     if (m == 1) return 1;
48     if (l == r) return ext_mod(a[l], m);
49     return ext_pow(a[l], solve_tower(a, l + 1, r, get_phi(m)), m);
50 }
51 // a[l] ^ (a[l+1] ^ (a[l+2] ... ^ (a[r]) ) )
52 ll power_tower(const vector<ll> &a, int l, int r, ll m) {
53     return solve_tower(a, l, r, m) % m;
54 }

```

Stress

```

1  #include "bits/stdc++.h"
2
3  using namespace std;
4  #define ll long long
5  #define int long long
6
7  random_device rd;
8  mt19937_64 mt(rd());
9
10 ll rnd(ll l, ll r) { return uniform_int_distribution<ll>(l, r)(mt); }
11
12 const int LIMIT = 1e6;
13
14 void generate() {

```

```

15  ofstream cout("test.txt");
16
17  int n = rnd(1, 10);
18  cout << n << '\n';
19  for (int i = 0; i < n; ++i) {
20      cout << rnd(1, 1e5) << ' ';
21  }
22
23  cout.close();
24  }
25
26  int32_t main() {
27      system("g++ -lm -O3 -std=c++17 -DLOCAL -pipe -o main ../main.cpp");
28      system("g++ -lm -O3 -std=c++17 -pipe -o brute ../brute.cpp");
29      // system("g++ -lm -O3 -std=c++17 -pipe -o gen ../gen.cpp"); // run gen file ?
30
31      for (int tc = 1; tc <= LIMIT; ++tc) {
32          cerr << "Case_" << tc << '\n';
33
34          // system("gen >test.txt"); /// file
35          // generate(); /// function
36
37          system("main <test.txt >wa.txt");
38          if (system("brute <test.txt >ac.txt")) break;
39          ifstream acs("ac.txt");
40          ifstream was("wa.txt");
41          string ac, wa;
42          getline(was, wa, (char) EOF);
43          getline(acs, ac, (char) EOF);
44          was.close();
45          acs.close();
46          if (ac != wa) break;
47      }
48  }

```